

Algoritmos e Programação I

Prof. Dr. Amaury Antônio de
Castro Junior

Módulo 3 - Variáveis compostas: *strings*, vetores e matrizes

Unidade 1 - Definição e manipulação de *strings*

O que já estudamos?

- Até aqui, já estudamos como tomar decisões em programas em *Python*, usando a **lógica** para a escrita de **expressões condicionais**, bem como estruturas e comandos para **controle do fluxo de execução** de programas.
- Para que o potencial das estruturas de controle possa ser plenamente explorado, também é necessário conhecer formas de armazenamento e manipulação de outros tipos de dados mais complexos e não apenas de dados simples.

Manipulando tipos de dados compostos

- Os tipos de dados manipulados por um algoritmo podem ser classificados em dois grupos: simples (atômicos), estudados até aqui, ou compostos (complexos).
- **Tipos simples:** são dados indivisíveis, tais como o tipo numérico, pois os números não são compostos de partes.
- **Tipos compostos:** são conjuntos de dados ou valores que podem ser decompostos em elementos (partes) mais simples.
- Exemplos: *strings*, vetores e matrizes, que estudaremos nesta aula.

Strings

- Entre os tipos complexos, vamos apresentar inicialmente apenas o tipo ***string***, cujo conjunto de valores é formado por frases ou cadeias de caracteres.
- Uma *string* é um conjunto de valores do tipo caracter.
- As *strings* aparecem, geralmente, entre aspas simples ou duplas (em *Python*).
- Costuma-se usar aspas simples para representar um único caractere e aspas duplas para *string*.

Acesso e alteração de conteúdo

- O acesso é direto.
- Use o nome da variável para o conteúdo completo da variável:

```
nome = "Carlo Souza"
print(nome)
```
- Os caracteres de uma *string* também podem ser acessados individualmente, pelo índice da posição:
 - OBS: A primeira posição tem índice zero.
- Sintaxe de Acesso: use o índice entre colchetes
 - `nome[0]` retorna "C"
 - `nome[6]` retorna "S" da palavra "Souza".

	0	1	2	3	4	5	6	7	8	9	10
nome	C	A	R	L	O		S	O	U	Z	A

Detalhes de operação e manipulação

- *Strings* são **imutáveis**: conteúdo de uma posição específica não pode ser modificado diretamente.

```
nome = "João Silva"
```

```
nome[3] = "t"
```

`TypeError: 'str' object does not support item assignment`

- Operadores úteis:

Operador	Descrição
<code>in</code>	verifica se uma <i>substring</i> existe dentro da <i>string</i>
<code>len</code>	retorna o tamanho da <i>string</i> (número de caracteres)
<code>+</code>	concatena duas <i>strings</i>
<code>*</code>	repete a <i>string</i> o número de vezes indicado

Exemplos de uso dos operadores

- Considerando que a variável `nome` é inicializada da seguinte forma:

```
nome = "Maria Silva"
```

- Qual será a saída dos comandos de impressão abaixo:

```
print("M" in nome)  
print("B" in nome)  
print("ria" in nome)  
print("m" in nome)  
print(len("Maria"))  
print(len(nome))
```


Exemplos de uso dos operadores

- Considerando que a variável `nome` é inicializada da seguinte forma:

```
nome = "Maria Silva"
```

- Qual será a saída dos comandos de impressão abaixo:

```
print("M" in nome) True
print("B" in nome) False
print("ria" in nome) True
print("m" in nome) False
print(len("Maria")) 5
print(len(nome)) 11
```

Exemplos de concatenação e multiplicação

- Considerando que as variáveis abaixo foram inicializadas como segue:

```
nome = "Maria"  
sobrenome = "Silva"
```

- Qual será a saída dos comandos de impressão abaixo:

```
nome_completo = nome + sobrenome  
print(nome_completo)  
print(nome * 2)
```

Exemplos de concatenação e multiplicação

- Considerando que as variáveis abaixo foram inicializadas como segue:

```
nome = "Maria"  
sobrenome = "Silva"
```

- Qual será a saída dos comandos de impressão abaixo:

```
nome_completo = nome + sobrenome  
print(nome_completo) MariaSilva  
print(nome * 2) MariaMaria
```

Percorrendo caracteres de uma *String*

Usando *while*:

```
nome = "Maria Silva"  
i = 0  
while i < len(nome):  
    print(nome[i])  
    i = i + 1
```

Usando *for*:

```
nome = "Maria Silva"  
for i in range(len(nome)):  
    print(nome[i])
```

- O laço "***while***" é mais simples de entender e compreender como está sendo percorrida a *string*.
- O laço "***for***" é mais conciso, pois não precisa inicializar e nem incrementar manualmente o contador.

Métodos úteis com *String*

Operador	Descrição
<code>upper</code>	converte os caracteres da <i>string</i> para maiúsculas
<code>lower</code>	converte os caracteres da <i>string</i> para minúsculas
<code>split</code>	divide a <i>string</i> em partes em pontos específicos, considerando o caractere separador especificado
<code>partition</code>	divide a <i>string</i> em três partes, considerando a primeira ocorrência do caractere separador especificado

Exemplos de uso: *upper/lower*

- Considerando o conteúdo das variáveis abaixo, o que será impresso na tela por cada comando de impressão?

```
texto = "Quem parte e reparte, fica com a maior parte"  
texto_up = texto.upper()  
print(texto_up)  
  
print(texto)  
  
texto_lw = texto.lower()  
print(texto_lw)
```

Exemplos de uso: *upper/lower*

- Considerando o conteúdo das variáveis abaixo, o que será impresso na tela por cada comando de impressão?

```
texto = "Quem parte e reparte, fica com a maior parte"  
texto_up = texto.upper()  
print(texto_up)
```

QUEM PARTE E REPARTE, FICA COM A MAIOR PARTE

```
print(texto)
```

Quem parte e reparte, fica com a maior parte

```
texto_lw = texto.lower()  
print(texto_lw)
```

quem parte e reparte, fica com a maior parte

Exemplos de uso: *split/partition*

- O método ***split*** separa a *string* em partes usando o separador especificado.
- Se não especificado, usa espaços, *tabs* e quebras de linha como separador padrão.
- Muito útil para ler uma lista de valores de entrada com ***input***:

```
x, y = input("Digite dois valores: ").split()
```

```
print(x)
```

```
print(y)
```


Exemplos de uso: *split/partition*

- O método ***split*** separa a *string* em partes usando o separador especificado.
- Se não especificado, usa espaços, *tabs* e quebras de linha como separador padrão.
- Muito útil para ler uma lista de valores de entrada com ***input***:

```
x, y = input("Digite dois valores: ").split()
```

```
Digite dois valores: 10 20
```

```
print(x)
```

```
"10"
```

```
print(y)
```

```
"20"
```

Exemplos de uso: *split/partition*

- É possível especificar um separador no método ***split***, quando necessário.
- Por exemplo na leitura de uma data, usando a "/" para quando queremos separar dia mês e ano:

```
dia, mes, ano = input("Digite uma data: ").split("/")
```

```
print(dia)
```

```
print(mes)
```

```
print(ano)
```

Exemplos de uso: *split/partition*

- É possível especificar um separador no método *split*, quando necessário.
- Por exemplo na leitura de uma data, usando a "/" para quando queremos separar dia mês e ano:

```
dia, mes, ano = input("Digite uma data: ").split("/")
```

```
Digite uma data: 10/04/2018
```

```
print(dia)
```

```
"10"
```

```
print(mes)
```

```
"04"
```

```
print(ano)
```

```
"2018"
```

Exemplos de uso: *split/partition*

- O método ***partition*** separa a *string* em três partes usando a primeira ocorrência do separador especificado.
- Útil para processar elementos um por vez.
- Exemplo:

```
antes, sep, depois = input("Digite valores: ").partition("-")
```

```
print(antes)
```

```
print(sep)
```

```
print(depois)
```

Exemplos de uso: *split/partition*

- O método *partition* separa a *string* em três partes usando a primeira ocorrência do separador especificado.
- Útil para processar elementos um por vez.
- Exemplo:

```
antes, sep, depois = input("Digite valores: ").partition("-")
Digite valores: 10-20-30-40
print(antes)
"10"
print(sep)
"_"
print(depois)
"20-30-40"
```

Exercício resolvido 1

- Faça um programa em *Python* que leia uma palavra e, em seguida, uma letra qualquer do alfabeto. Seu programa deve apresentar uma mensagem dizendo quantas vezes essa letra aparece na palavra.

Exercício resolvido 1

- Faça um programa em *Python* que leia uma palavra e, em seguida, uma letra qualquer do alfabeto. Seu programa deve apresentar uma mensagem dizendo quantas vezes essa letra aparece na palavra.
- Problema:
 - Ler uma palavra e uma letra.
 - Contar quantas vezes a letra aparece na palavra.
 - Exemplo: palavra = "bananinha", letra = "a" → resultado: 3 vezes.
- Veremos uma ideia de solução passo a passo, usando o que estudamos nesta aula e em aulas anteriores.

Exercício resolvido 1

Passo 1: Ler os dados de entrada informados pelo usuário

```
palavra = input("Digite uma palavra: ")  
letra = input("Digite uma letra: ")
```

A função **input()** captura a palavra e a letra digitadas pelo usuário no teclado.

Exercício resolvido 1

Passo 2: Contar as ocorrências da letra, percorrendo a palavra (laço com índice e contador)

```
i = 0
contador = 0

while i < len(palavra):
    if palavra[i] == letra:
        contador = contador + 1
    i = i + 1
```

Exercício resolvido 1

Passo 2: Contar as ocorrências da letra, percorrendo a palavra (laço com índice e contador)

```
i = 0
contador = 0

while i < len(palavra):
    if palavra[i] == letra:
        contador = contador + 1
    i = i + 1
```

- Usa índice **i** para acessar cada caractere da palavra (limite do tamanho da palavra) e comparar com a letra informada pelo usuário.

Exercício resolvido 1

Passo 2: Contar as ocorrências da letra, percorrendo a palavra (laço com índice e contador)

```
i = 0
contador = 0

while i < len(palavra):
    if palavra[i] == letra:
        contador = contador + 1
    i = i + 1
```

- Compara cada caractere com a letra procurada, informada pelo usuário.

Exercício resolvido 1

Passo 2: Contar as ocorrências da letra, percorrendo a palavra (laço com índice e contador)

```
i = 0
contador = 0

while i < len(palavra):
    if palavra[i] == letra:
        contador = contador + 1
    i = i + 1
```

- Se forem iguais, incrementa o contador de ocorrências.

Exercício resolvido 1

Passo 3: Escrever o resultado na tela:

```
print(f"A letra {letra} aparece {contador} vezes na palavra {palavra}")
```

Exercício resolvido 1

Código completo (para testar):

```
palavra = input("Digite uma palavra: ")
letra = input("Digite uma letra: ")
i = 0
contador = 0

while i < len(palavra):
    if palavra[i] == letra:
        contador = contador + 1
    i = i + 1

print(f"A letra {letra} aparece {contador} vezes na palavra {palavra}")
```

Exercício resolvido 2

- Faça um programa em *Python* que leia uma frase e, em seguida, escreva a quantidade de caracteres da frase, ignorando os espaços em branco. No final, mostre também a frase original digitada pelo usuário.

Exercício resolvido 2

- Faça um programa em *Python* que leia uma frase e, em seguida, escreva a quantidade de caracteres da frase, ignorando os espaços em branco. No final, mostre também a frase original digitada pelo usuário.
- Problema:
 - Ler uma frase e contar quantos caracteres ela tem, ignorando espaços em branco.
 - Exemplo: "Olá mundo!" → 9 caracteres (O,l,á,m,u,n,d,o,!).

Exercício resolvido 2

Passo 1: Ler a frase informada pelo usuário

```
frase = input("Digite uma frase: ")
```

A função **input()** captura a frase digitada pelo usuário no teclado.

Exercício resolvido 2

Passo 2: Contar apenas caracteres NÃO espaço

```
p = 0
contador_sem_espaco = 0

while p < len(frase):
    if frase[p] != " ":
        contador_sem_espaco = contador_sem_espaco + 1
    p = p + 1
```

- Percorre cada posição com índice **p**.

Exercício resolvido 2

Passo 2: Contar apenas caracteres NÃO espaço

```
p = 0
contador_sem_espaco = 0

while p < len(frase):
    if frase[p] != " ":
        contador_sem_espaco = contador_sem_espaco + 1
    p = p + 1
```

- Se o caractere não for espaço (`!= " "`), conta 1. Só devo contar o que for diferente de espaço.

Exercício resolvido 2

Passo 2: Contar apenas caracteres NÃO espaço

```
p = 0
contador_sem_espaco = 0

while p < len(frase):
    if frase[p] != " ":
        contador_sem_espaco = contador_sem_espaco + 1
    p = p + 1
```

- Dessa forma, o programa ignora os espaços automaticamente, contando apenas os caracteres que são diferentes do espaço.

Exercício resolvido 2

Passo 3: Mostra o resultado da contagem e a frase original na tela

```
print(f"A frase tem {contador_sem_espaco} caracteres sem os espaços.")  
print(f"Frase original: {frase}")
```

Exercício resolvido 2

Código completo (para testar):

```
frase = input("Digite uma frase: ")
p = 0
contador_sem_espaco = 0

while p < len(frase):
    if frase[p] != " ":
        contador_sem_espaco = contador_sem_espaco + 1
    p = p + 1

print(f"A frase tem {contador_sem_espaco} caracteres sem os espaços.")
print(f"Frase original: {frase}")
```

Exercício resolvido 3

- Faça um programa em *Python* que leia uma palavra informada pelo usuário e, em seguida, escreva a palavra invertida (de trás para frente), utilizando somente letras maiúsculas.

Exercício resolvido 3

- Faça um programa em *Python* que leia uma palavra informada pelo usuário e, em seguida, escreva a palavra invertida (de trás para frente), utilizando somente letras maiúsculas.
- Problema:
 - Ler uma palavra informada pelo usuário e mostrá-la invertida em maiúsculas.
 - Exemplo: "alfabeto" → "OTEBAFLA".

Exercício resolvido 3

Passo 1: Ler a palavra informada pelo usuário

```
palavra = input("Digite uma palavra: ")
```

A função **input()** captura a palavra digitada pelo usuário no teclado.

Exercício resolvido 3

Passo 2: Converter as letras da palavra para maiúsculas

```
palavra_maiuscula = palavra.upper()
```

- `upper()` é um método de *strings* em Python
- Ele devolve uma nova *string* com todas as letras em maiúsculas
- Ex.: `"Python".upper()` resulta em `"PYTHON"`

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Começamos com `palavra_invertida` vazia ("")
- O laço `for` percorre cada letra da string `palavra_maiuscula`
- Em cada passo, colocamos a letra na frente do que já está em `palavra_invertida`

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Exemplo de construção:
- `palavra_maiuscula = "CASA"`
- `palavra_invertida = ""`
- Concatena a letra C $\rightarrow$ `palavra_invertida = "C"`

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Exemplo de construção:
- palavra_maiuscula = "CASA"
- palavra_invertida = "C"
- Concatena a letra A → palavra_invertida = "AC"

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Exemplo de construção:
- `palavra_maiuscula = "CASA"`
- `palavra_invertida = "AC"`
- Concatena a letra S → `palavra_invertida = "SAC"`

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Exemplo de construção:
- palavra_maiuscula = "CASA"
- palavra_invertida = "SAC"
- Concatena a letra A → palavra_invertida = "ASAC"

Exercício resolvido 3

Passo 3: Construir a palavra invertida com um laço

```
palavra_invertida = ""
```

```
for letra in palavra_maiuscula:  
    palavra_invertida = letra + palavra_invertida
```

- Exemplo de construção:
- `palavra_maiuscula = "CASA"`
- `palavra_invertida = "ASAC"`
- No final do laço, `palavra_invertida` contém a palavra original ao contrário.

Exercício resolvido 3

Passo 4: Escrever a palavra invertida na tela

```
print(f"Palavra invertida: {palavra_invertida}")
```

- A função `print()` mostra o texto na tela.
- Exibe a mensagem "Palavra invertida: " seguida da palavra invertida em maiúsculas que foi construída no passo anterior.

Desafio

- Faça um programa que leia uma data de nascimento no formato **dd/mm/aaaa** e imprima a data com o mês escrito por extenso.

Exemplo:

Data = 20/02/1995

Resultado gerado pelo programa:

Você nasceu em 20 de fevereiro de 1995

Resumo da aula

- Vimos a diferença entre tipos simples e compostos e começamos com o estudo das **strings** (cadeia de caracteres).
- Vimos também como definir e utilizar **strings** em Python.
- Foram apresentadas alguns operadores, operações, dicas de uso e exemplos de manipulação de **strings**.
- Foram apresentados alguns exercícios e suas respectivas soluções como forma de ilustrar algumas aplicações simples para o uso de **strings**.

Dicas e próximos passos

- Implemente e teste os códigos apresentados nessa aula em IDEs e interfaces interativas:
 - IDEs: Thonny, PyCharm, VSCode;
 - Interfaces interativas: IDLE, Jupyter Notebook, Google Colab.
- Aprender algoritmos exige empenho, dedicação e muitos muitos exercícios;
- Leia os materiais indicados como referências;
- Busque outras fontes;
- Faça pesquisas e exercícios;
- Comprometa-se com o curso e a disciplina.

Referências

BANIN, S. L. **Python 3: Conceitos e Aplicações - Uma Abordagem Didática**. São Paulo: Erica, 2018. ISBN 9788536530253. **Disponível na Biblioteca Digital da UFMS.**

MANZANO, J. A. N. G. **Algoritmos: Lógica para Desenvolvimento de Programação de Computadores**. 29 ed. São Paulo: Erica, 2019. ISBN 9788536531472. **Disponível na Biblioteca Digital da UFMS.**

PERKOVIC, L. **Introdução à computação usando Python: um foco no desenvolvimento de aplicações**. Rio de Janeiro: LTC, 2016. ISBN 9788521630937. **Disponível na Biblioteca Digital da UFMS.**

WENTWORTH, P.; ELKNER, J.; DOWNEY, A. B.; MEYERS, C. **How to think like a computer scientist: Learning With Python 3**. [S.L.: S. N.], 2012. Disponível em: <https://link.ufms.br/Vow0N>. Acesso em 05 mai. 2025.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

